



aurelia:
an ARPES spectra simulator
for training deep-learning algorithms

M. X. Na^{*,1,2}, M. Michiardi^{*,1,2}, A. Damascelli^{1,2}

¹Department of Physics and Astronomy, University of British Columbia
Vancouver, BC, V6T 1Z1, Canada

²Quantum Matter Institute, Vancouver, BC, V6T 1Z4, Canada

Contents

1	Module 1: aurelia_arpes	4
1.1	Class: Bands	4
1.1.1	Initialization	4
1.1.2	Define k-path	5
1.1.3	Calculate bands	5
1.1.4	Importing bands from an external file	6
1.1.5	Print variables	7
1.2	Class: Spec	7
1.2.1	Initialization	7
1.2.2	Define the self-energy	8
1.2.3	Photoemission matrix elements	9
1.2.4	Calculate the spectral function	10
1.2.5	Add resolution broadening	11
1.2.6	Print variables	12
1.3	Class: ARPES	12
1.3.1	Initialization	12
1.3.2	Momentum-to-angle conversion:	13
1.3.3	Checking the warping	15
1.3.4	Creating statistics	15
1.3.5	Quality scoring	16
1.3.6	Print variables	16
2	Module 2: aurelia_static_vars	17
2.1	Class: Modifier	17
2.1.1	Initialization	17
2.2	Class: Domain	17
2.2.1	Initialization	17
2.2.2	Make rotational domain	18
2.2.3	Remove rotational domains	19
2.2.4	Make flake domain	19
2.2.5	Remove flake domains	20
2.3	Class: Experiment	20
2.3.1	Initialization	20
2.3.2	Adding background	22
2.3.3	Adding responsivity	23
3	Module 3: aurelia_plots	23
3.1	Class: show_spectra	23
3.1.1	Plot spectral function and bands	24

3.1.2	Plot ARPES spectra	24
3.1.3	Plot domains	24
3.1.4	Plot statistical simulation	24
4	Module 4: aurelia_hdf5_saving	24
4.1	Class: DictGroup	25
4.1.1	convert_strings_to_utf8(obj)	25
4.1.2	save_dict_to_hdf5	25
4.1.3	save_class_to_hdf5	25
4.1.4	Example file structure	26
5	Example code	27
5.1	Simulating spectra	27
5.2	Adding domains	29
5.3	Adding quality score	30
6	Appendix	31
6.1	Momentum to angle conversion details:	31

Introduction

Welcome to the *aurelia* package developed by MengXing Na and Matteo Michiardi at the University of British Columbia. This package has been developed with the intention of simulating spectra from angle-resolved photoemission Spectroscopy (ARPES) experiments which contain all manner of experimental artifacts for the purpose of training machine-learning algorithms. Here you will find all relevant modules and libraries for running calculations with this package.

For any questions which you do not find explanations in the documentation files, please do not hesitate to contact me at mengxing.na@gmail.com

1 Module 1: aurelia_arpes

This module contains the building blocks of the simulation. It includes three objects: Bands, Spec, and ARPES:

- The object Bands defines the electronic dispersion. The in-plane k-path – specified by a list of k_x and k_y points – and the band dispersion is calculated along the path according to a specified tight-binding model. Alternatively, one can also import the band dispersion from an external file.
- The object Spec calculates the ARPES spectra one would expect from the dispersion. First, one calculates the one-electron removal spectral function from the electronic dispersion and the self-energy. Then, the photoemission matrix elements are defined. Lastly, one defines the Fermi-Dirac distribution given an effective electronic temperature, and convolves the final result with a Gaussian for resolution broadening.
- The object ARPES gives the simulation of ARPES data one would collect. It transforms Spec, calculated in crystal momentum and binding energy $(\mathbf{k}, \omega)$ into emission angles and kinetic energy (θ, ϕ, E_k) .

1.1 Class: Bands

1.1.1 Initialization

```
def __init__(self, symmetry=None, klim=None, Nbands=None, warp=None, Npts=None, edges=None):
```

The inputs are described below. Note that if these inputs are not given, one is usually chosen at random. This is because we want to randomly generate dispersions and use them to train deep-learning algorithms. Here, variety matters more than accuracy. However, if accuracy is desired, one can import the band dispersion itself from an external file, (for example from a DFT calculation or tight-binding) in which case the default inputs will be fetched from the file itself and overwritten with the correct values (see Sec. 1.1.4).

- `symmetry`: A string. Defines the lattice of the tight-binding model. The available inputs are ‘rectangle’, ‘square’, ‘hexagonal’, ‘honeycomb’. If an input is not given, one is chosen at random.
- `klim`: A 2×2 numpy array defining the limits of k_x and k_y in the list. If an input is not given, the lower limit is chosen randomly between $[-1, -0.5]$, and the upper limit is chosen randomly between $[0.5, 1]$.
- `Nbands`: An integer defining the number of bands in the tight-binding model. If an input is not given, one is chosen at random between $[1, 5]$.
- `warp`: A string. "on" or "off". Turns on and off nearest neighbour hopping, or warping of the bands.
- `Npts`: A 2×1 numpy array defining the number of points calculated in the k_x and k_y dimensions. The total number of points is given by `np.prod(Npts)`.
- `edges`: A float slightly larger than one. Since we calculate a limited region of k -space, interpolation may be off at the edges if we use large offset angles in the experiment. Here, the k -space edge is padded to crop out the edge effects after calculation.

1.1.2 Define k-path

```
def Make_kpath(self):
```

This function defines the k_x and k_y grid the bands are calculated on. The edges are padded so that rotation and interpolation do not create artifacts. The calculation of the dispersion is not particularly time-intensive, so a grid of points in k_x and k_y are typically calculated. To speed up computation, the ARPES spectra can be calculated over just k_x , but the grid of points are needed for interpolation in the case of rotational or offset domains.

This function adds the following to the `Bands(self)` object:

- `self.kpath`: A $N \times 2$ numpy array defining the k -path along which the dispersion is calculated. N is given by `np.prod(Npts)`. The first column defines k_x , the second column is k_y .
- `self.kax`: A $N_{pts}^x \times 1$ numpy array defining the k_x axis. Used for plotting purposes.
- `self.kay`: A $N_{pts}^y \times 1$ numpy array defining the k_y axes. Used for plotting purposes.

1.1.3 Calculate bands

```
def Make_bands(self, tb=None, lattice=None):
```

This function calculates the dispersion using a tight-binding model. The inputs are as follows:

- `tb`: A Python dictionary of the form `tb = {"E0": [], "t": []}`. `E0` and `t` must be numpy arrays of dimension $1 \times N_{\text{bands}}$. `E0` are the centre energies of the bands, and `t` is the hopping parameter of each band. If the lattice is rectangular, as defined in `Bands.symmetry`, then `t` must be an array of dimension $[2, N_{\text{bands}}]$. If the input is not given, `E0` will be randomly generated on the interval $[-1, 0]$ and `tb` will be randomly generated on the interval $[-1, 1]$.
- `lattice`: A Python dictionary of the form `lattice = {"a": [], "b": []}`. These lattice parameters define the size of the Brillouin zone and – by extension – the field of view given by the limits set in `Bands.klim`.

The tight-binding dispersions for the following lattices are given. The E_{nn} term is added if `self.warp = "on"`

$$\begin{aligned}
\text{Square : } \quad E_{\text{n}} &= E_0 + 2t[\cos(k_x a) + \cos(k_y a)] \\
E_{\text{nn}} &= rt[\cos(2k_x a) + \cos(2k_y a)] \\
\text{Rectangle : } \quad E_{\text{n}} &= E_0 + 2t_1 \cos(k_x a) + 2t_2 \cos(k_y b) \\
E_{\text{nn}} &= E_0 + rt_1 \cos(2k_x a) + rt_2 \cos(2k_y b) \\
\text{Hexagonal : } \quad E_{\text{n}} &= E_0 + 2t[\cos(a/2(k_x + \sqrt{3}k_y)) + \\
&\quad \cos(a/2(k_x - \sqrt{3}k_y)) + \cos(k_x a)] \\
E_{\text{nn}} &= E_0 + 2rt[\cos(a/2(3k_x + \sqrt{3}k_y)) + \\
&\quad \cos(a/2(\sqrt{3}k_y - 3k_x) + \cos(\sqrt{3}k_y a)] \\
\text{Honeycomb : } \quad E_{\text{n}} &= E_0 \pm \\
&\quad t\sqrt{3 + 2\cos(\sqrt{3}k_x a) + 4\cos(\sqrt{3}k_x a/2)\cos(3k_y a/2)} \\
E_{\text{nn}} &= 2rt[\cos(a/2(\sqrt{3}k_x + 3k_y)) + \\
&\quad \cos(a/2(\sqrt{3}k_x - 3k_y)) + \cos(a\sqrt{3}k_x)]
\end{aligned} \tag{1}$$

The outputs are as follows:

- `self.Bands`: A $N \times N_{\text{bands}}$ numpy array defining the dispersion of each band along the k -path. N is given by `np.prod(Npts)`.

1.1.4 Importing bands from an external file

`def Import_bands(self, filename):`

This function imports bands from a file. The file should have values of k_x in column 1, k_y in column 2. Each band should be in its own column from 3 onward. The following parameters are read in from the file:

- `Npts, kax, kay, klim, kpath, bands, Nbands`

The following parameters are defined statically:

- `self.edges = 1`
- `self.warp = "off"`
- `self.symmetry = "custom"`

The file `"Graphene_bands.csv"` is given as an example.

1.1.5 Print variables

```
def print_variables(self):
```

This function provides a summary of variables used to calculate the band structure.

1.2 Class: Spec

The object `Spec` calculates the spectral intensity one would expect from the dispersion. First, one calculates the one-electron removal spectral function from the electronic dispersion and the self energy. Then, the photoemission matrix elements are defined. Lastly, one defines the Fermi-Dirac distribution given an effective electronic temperature, and convolves the final image with Gaussian functions for resolution broadening.

1.2.1 Initialization

```
def __init__(self, bands, dimension=None, Omega=None, ME=None):
```

The exposed inputs are described below. Note that if these inputs are not given, one is usually chosen at random.

- `bands`: An object. Defines the electronic dispersion and k -path.
- `dimension`: A string. Defines the dimension of the calculation, three modes are available. The input `"cube"` calculates the spectra in binding energy ω , and the crystal momenta k_x , and k_y . The input `"sliceEk"` calculates the spectra in ω and k_x . The so-called “dispersion cut”. The input `"sliceek"` calculates the spectra in k_x and k_y , the so-called “Fermi surface” or “constant energy cut”. These modes are defined for computational time and file size considerations.
- `Omega`: A $M \times 1$ numpy array defining the binding energy range points over which the spectra are calculated.

The following outputs are added to initialize the calculation:

- `self.bands`: The object defining the electronic dispersion and k -path.

- `self.kax`: A $N_{pts}^x \times 1$ numpy array defining the k_x axis. Used for plotting purposes.
- `self.kay`: A $N_{pts}^y \times 1$ numpy array defining the k_y axes. Used for plotting purposes.
- `self.matrix_elements`: A $N \times N_{bands}$ numpy array defining the matrix elements of each band along the k -path. N is given by `np.prod(Npts)`. This is the same shape as `Bands.bands`. These are initialized as an array of ones.
- `self.ReS`: A $M \times N_{bands}$ numpy array defining the real part of the self energy. These are initialized as zeros. M is the length of `self.Omega`.
- `self.ImS`: A $M \times N_{bands}$ numpy array defining the imaginary part of the self energy. These are initialized with a single value of 0.2. M is the length of `self.Omega`.
- `self.domain`: A numpy array defining the relative intensity of rotational and offset domains. Initialized (for the primary domain) as an array of ones with dimension $1 \times N_{bands}$.

1.2.2 Define the self-energy

`def Make_self_energy(self, SE):`

This functional is optional, as the self energy is already initialized. The real part is initialized as zero, and the imaginary part is initialized as a constant value of 0.2. However, for more realistic ARPES spectra, we can define two types of self-energy here.

The exposed input `SE` is a Python dictionary of the form `SE = {"type": [], ...}`. The input "type" accepts two strings.

- 'FL': This stands for Fermi-liquid self energy. Here, the real part of the self energy is zero, and the imaginary part of the self energy is quadratic in ω , multiplied by a constant given by the number D , that is:

$$\begin{aligned}\Sigma' &= 0 \\ \Sigma'' &= D\omega^2 + \Sigma_0\end{aligned}\tag{2}$$

where we additionally define in the dictionary `SE["val"] = D` and `SE["ImS0"] =  $\Sigma_0$` . If these values are not given, they are chosen as follows:

- $\Sigma_0 = 0.01$. This prevents the band from being too sharp and creating artifacts at the Fermi-level during the calculation.
- $D \in [0.05, 0.25]$.

- 'kink': This stands for a self-energy featuring an electron-boson kink. The self-energy $\Sigma = \Sigma' + i\Sigma''$ is phenomenological, and calculated as follows:

$$\begin{aligned}\Sigma' &= R \left[\frac{\gamma}{(\omega + \omega_0)^2 + \gamma^2} - \frac{\gamma}{(\omega - \omega_0)^2 + \gamma^2} \right] \\ \Sigma'' &= \Sigma_1 - \frac{\Sigma_1 - \Sigma_0}{1 + e^{-(\omega + \omega_0)\gamma}}\end{aligned}\tag{3}$$

The parameters for this calculation are exposed and can be customized using the dictionary keys of SE as follows:

- `SE["Amp"] = R`, `SE["Ekink"] =  $\omega_0$` , `SE["gamma"] =  $\gamma$` ,
- `SE["ImS0"] =  $\Sigma_0$` , `SE["ImS1"] =  $\Sigma_1$`

In the event that these values are not explicitly defined by the user, they selected randomly as follows:

- $R \in [0, 0.01]$, $\omega_0 \in [0.05, 0.3]$, $\gamma \in [0, 50]$
- $\Sigma_0 \in [0.015, 0.025]$, $\Sigma_1 \in [0.115, 0.125]$

1.2.3 Photoemission matrix elements

`def Make_matrix_elements(self, ME=None):`

The matrix is initialized as a numpy array of ones, and therefore this function is optional. We note that this function is not a real calculation of the photoemission matrix elements, but rather, it aims to add intensity variation to the simulation in a way that mimics how real photoemission matrix elements affect the measured spectra.

The input ME is A Python dictionary of the form `ME = {"type": [], ...}`. Which defines the type of fake photoemission matrix elements that are generated. The input "type" accepts three strings, either singly or in combination. These are: "symm", "rot", and "poly". Note that if a combination is used, the strings must be enclosed in square brackets (i.e. they are a python list). The three types of intensity randomization is described below:

- "symm" gives the matrix elements the same symmetry as that of the tight-binding model, in that they are periodic in k -space in the same way. The equations are:

$$\begin{aligned}\text{Square : } M_n &= t[\cos(k_x a) + \cos(k_y a)] + P(k_x, k_y) \\ \text{Rectangle : } M_n &= t_1 \cos(k_x a) + 2t_2 \cos(k_y b)] + P(k_x, k_y) \\ \text{Hexagonal : } M_n &= t[\cos(a/2(k_x + \sqrt{3}k_y)) + \\ &\quad \cos(a/2(k_x - \sqrt{3}k_y)) + \cos(k_x a)] + P(k_x, k_y) \\ \text{Honeycomb : } M_n &= t\sqrt{3 + 2\cos(\sqrt{3}k_x a) + 4\cos(\sqrt{3}k_x a/2)\cos(3k_y a/2)} + P(k_x, k_y)\end{aligned}\tag{4}$$

If `Bands.symmetry = "custom"` from imported bands, then this mode does not do anything.

- "rot" gives rotational symmetry, which mimics the orbital selectivity of linear and/or circularly polarized light. i.e.:

$$\phi = \arctan\left(\frac{k_y}{k_x}\right), \quad M_{\text{rot}} = \cos(n\phi) \quad (5)$$

If this mode is selected, then n (specified in the field `ME["rotN"]`) gives the order of rotational symmetry. If `ME["rotN"]` is not specified, then the order is randomly selected according to the rotational symmetry of the lattice defined in the tight-binding model:

- Rectangle: $n = 1, 2$
- Square: $n = 1, 2, 4$
- Hexagonal/Honeycomb: $n = 1, 2, 3, 6$
- "poly" gives a random polynomial used to modulate the intensity of the bands and is used to mimic the unpredictable effects of experimental geometry and the spatial/spectral variance in the light. If this mode is selected, then the field "polyN" gives the order of the polynomial. If "polyN" is not specified, then the order is randomly selected.

The outputs are as follows:

- `self.matrix_elements`: A $N \times N_{\text{bands}}$ NumPy array defining the matrix elements M_n of the n bands along the k -path. N is given by `np.prod(Npts)`.
- `self.ME`: The Python dictionary specifying parameters of the matrix-element calculation.

1.2.4 Calculate the spectral function

```
def Make_specfun(self, mod):
```

This function calculates the spectral function based on the band structure as well as the self-energy given. For computational efficiency, the matrix elements are included here, even though they more appropriately belong under the ARPES object.

The input `mod` is a Python dictionary of the form `mod = {ER: [], kR: [], Temp: []}`, where the three fields define the energy resolution, the angular resolution, and the electronic temperature, respectively. `mod` is an object generated by the `aurelia_static_vars` module. See Sec. 2.1.

The intensity is calculated as:

$$\begin{aligned}
\text{cube: } I(k_x, k_y, \omega) &= \sum_m M_m(\omega) \frac{\Sigma_m''(\omega)}{(\omega - \epsilon_{\mathbf{k}} - \Sigma_m'(\omega))^2 + \Sigma_m''(\omega)^2} \\
\text{sliceEk: } I(k_x, k_{y0}, \omega) &= \sum_m M_m(\omega) \frac{\Sigma_m''(\omega)}{(\omega - \epsilon_{\mathbf{k}} - \Sigma_m'(\omega))^2 + \Sigma_m''(\omega)^2} \\
\text{slicekk: } I(k_x, k_y) &= \sum_m \int_{\Delta\omega} M_m(\omega) \frac{\Sigma_m''(\omega)}{(\omega - \epsilon_{\mathbf{k}} - \Sigma_m'(\omega))^2 + \Sigma_m''(\omega)^2} d\omega \\
\Delta\omega &= k_B T + \Delta E
\end{aligned} \tag{6}$$

The outputs of the function are:

- `self.mod`: The python dictionary which collects the modification parameters
- `self.specfun`: A numpy array of dimension:
 - `cube`: $M \times N_{pts}^x \times N_{pts}^y$
 - `sliceEk`: $M \times N_{pts}^x$
 - `slicekk`: $1 \times N_{pts}^x \times N_{pts}^y$

1.2.5 Add resolution broadening

`def Make_specmod(self, mod):`

This function modifies the intensity in `self.specfun` by adding the Fermi-Dirac distribution and resolution broadening. The input `mod` is a Python dictionary of the form `mod = {ER: [], kR: [], Temp: []}`, where the three fields define the energy resolution, the angular resolution, and the electronic temperature, respectively. `mod` is generated by the *aurelia_static_vars* module. See Sec. 2.1.

The Fermi-Dirac is added by multiplication:

$$\begin{aligned}
I_{\text{FD}}(\mathbf{k}, \omega) &= f_{\text{FD}}(\omega, T) I(k_x, k_y, \omega), \\
f_{\text{FD}}(\omega, T) &= \frac{1}{1 + e^{\omega - E_F / k_B T}}.
\end{aligned} \tag{7}$$

And the resolution effects are added by convolving $I_{\text{FD}}(k_x, k_y, \omega)$ in energy and momenta with Gaussian function whose full-width-half-max are defined by ΔE and Δk , respectively.

$$I_{\text{res}}(\mathbf{k}, \omega) = G(\Delta k, k_x, k_y) * (G(\Delta E, \omega) * I_{\text{FD}}(\mathbf{k}, \omega)) \tag{8}$$

The outputs of the function are the following:

- `self.specfun`: The modified $I_{\text{res}}(\mathbf{k}, \omega)$ replaces the previously calculated $I(\mathbf{k}, \omega)$. This saves a bit on file-size.

1.2.6 Print variables

```
def print_variables(self):
```

This function provides a summary of variables used to calculate the spectral intensity.

1.3 Class: ARPES

The object ARPES takes the spectral intensity calculated by the object Spec as a function of binding energy and momentum, and converts it to kinetic energy and emission angle. Real experimental conditions, such as the presence of rotational and flake domains, the number of electrons collected, detector sensitivity, experimental background, etc. are also included, so that the final simulation closely resembles what may be measured in experiment under both optimal, and –more importantly– suboptimal conditions.

1.3.1 Initialization

```
def __init__(self, spec, exp, ang_lim=None, dimension=None):
```

The exposed outputs are described below:

- `spec`: An object containing the spectral intensity calculated from the dispersion.
- `exp`: An object containing the experimental parameters. For details, see Sec. 2.3. For the initialization, the photon energy `exp.hv` is used.
- `dimension`: A string. Defines the dimension of the calculation, three modes are available. The input "cube" calculates the spectra in binding energy, k_x , and k_y . The input "sliceEk" calculates the spectra in binding energy and k_x . The so-called “dispersion cut”. The input "slicekk" calculates the spectra in k_x and k_y , the so-called “Fermi surface”. The default dimension string is the same as the dimension of `Spec.dimension`, however, if `Spec.dimension = "cube"`, `arpes.dimension` can be "sliceEk" or "slicekk". This enables offset and rotational domains to be accurately calculated, while saving computational time as well as file size.
- `ang_lim`: Set the limits of θ and ϕ that the detector sees. If this is not specified, then the angle limits are calculated using the limits of k-space calculated in `Spec`. Details are included in the momentum-to-angle conversion in Sec. 1.3.2.

The outputs of the initialization are as follows:

- `self.spec`: An object containing the spectral intensity calculated from the dispersion.
- `self.exp`: An object containing the experimental parameters.
- `self.Ek`: The kinetic energy axis, calculated as:

$$E_k = h\nu - \Phi - \omega \quad (9)$$

- `self.dimension`: A string. Defines the dimension of the calculation.
- `self.th`: A numpy array of dimension $N_{pts}^x \times 1$. Defines the angles along the k_x direction.
- `self.ph`: A numpy array of dimension $N_{pts}^y \times 1$. Defines the angles along the k_y direction.

1.3.2 Momentum-to-angle conversion:

`def Make_angle_conv(self, const=None, domain=None):`

This function converts the $I_{\text{res}}(k_x, k_y, \omega)$ in `spec.specfun` to $I(\theta, \phi, E_k)$. The inputs are:

- `const`: A float. If the dimension of the calculation is not "cube", then `const` gives the value at which the slice is calculated. i.e.:
 - for `self.dimension = "sliceEk"`, `const = ky0`. Defaults to the mean k_y value.
 - for `self.dimension = "slicekk"`, `const = E0`. Defaults to the Fermi-energy.
- `domain`: An object containing information about offset angles for calculating offset domains.

The angular limits are set in `self.ang_lim`, if the values are not expressly defined, they are calculated to be as wide as possible given the momentum space and photon energy. Details of the conversion are given in the appendix (Sec. 6), briefly, we define the sample's out-of-plane direction as $\hat{z}$, then the axis that is parallel (perpendicular) to the slit of the hemispherical analyzer is denoted $\hat{y}$ ($\hat{x}$). We will define the polar angle θ to be a rotation about the $\hat{y}$ axis, the tilt angle ϕ to be a rotation about the $\hat{x}$ axis, and the azimuth angle α to be about the $\hat{z}$ axis. The θ and α can be calculated from the conservation of momentum:

$$\theta = \text{sign}(k_x) \arcsin \left(\frac{\hbar \sqrt{k_x^2 + k_y^2}}{\sqrt{2m_e E_k}} \right), \quad \alpha = \arcsin \left(\frac{\hbar k_y}{\sqrt{2m_e E_k} \sin \theta} \right) \quad (10)$$

Then the vector defining the direction of photoelectron propagation is:

$$\begin{aligned} d' &= R(\alpha, \hat{z}) R(\theta, \hat{y}) d \\ &= \begin{pmatrix} \cos \alpha \cos \theta & -\sin \alpha & \cos \alpha \sin \theta \\ \sin \alpha \cos \theta & \cos \alpha & \sin \alpha \sin \theta \\ -\sin \theta & 0 & \cos \theta \end{pmatrix} \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} \cos \alpha \sin \theta \\ \sin \alpha \sin \theta \\ \cos \theta \end{pmatrix} \end{aligned} \quad (11)$$

Then to get the angles of emission w.r.t. the analyzer, we simply need to find the θ_m and ϕ_m angles needed to map d' onto d , arranged such that we rotate along the analyzer slit (θ_m) first,

then perpendicular to it (ϕ_m).

$$d = R(\phi_m, \hat{x})R(\theta_m, \hat{y})d'$$

$$\begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} \cos \theta_m & 0 & \sin \theta_m \\ \sin \phi_m \sin \theta_m & \cos \phi_m & -\sin \phi_m \cos \theta_m \\ -\cos \phi_m \sin \theta_m & \sin \phi_m & \cos \phi_m \cos \theta_m \end{pmatrix} \begin{pmatrix} d'_x \\ d'_y \\ d'_z \end{pmatrix} \quad (12)$$

Adding the offset angles are rather trivial. We simply apply the rotation matrices $R(\theta_0, \hat{y})$ and $R(\phi_0, \hat{x})$ to d before calculating the photoemission angles. Such that

$$d' = R(\alpha, \hat{z})R(\theta, \hat{y})R(\phi_0, \hat{x})R(\theta_0, \hat{y})d \quad (13)$$

This then gives rise to a system of equations with which we solve for θ_m and ϕ_m , which gives:

$$\theta_m = -\arctan\left(\frac{d'_x}{d'_z}\right)$$

$$\phi_m = \arcsin(d'_y) \quad (14)$$

We can also apply the rotation perpendicular to the slit direction first, then along the slit direction. In this case, we simply reverse the order of the rotational matrices so that:

$$d = R(\theta_m, \hat{y})R(\phi_m, \hat{x})d' \quad (15)$$

In this case, the system of equations gives:

$$\theta_m = -\arcsin(d'_x)$$

$$\phi_m = \arctan\left(\frac{d'_y}{d'_z}\right) \quad (16)$$

Having obtained this conversion for all k_x and k_y , we can simply define a square mesh of angles θ_M and ϕ_M . and interpolate onto the mesh using the values of $(\theta_m, \phi_m, I_{\text{res}}(\mathbf{k}, \omega))$. The resulting values $I(\theta, \phi, E_k)$ are written into `arpes.intensity`. When simulating a sample with “flakey” domains at different offset angles, the conversion is done for each domain using this function. In this case, The outputs of the function are: $I(\theta, \phi, E_k, \theta_0, \phi_0)$ is not written into `arpes.intensity`, but is simply returned from the function. So intensity from all the flakes can be added outside the function.

- `self.intensity`: A numpy array of dimension:

- `cube`: $M \times N_{pts}^x \times N_{pts}^y$
- `sliceEk`: $M \times N_{pts}^x$
- `slicekk`: $1 \times N_{pts}^x \times N_{pts}^y$

1.3.3 Checking the warping

```
def Make_kwarp_check(self):
```

This function double checks the conversion from momentum to angle. First, the angles are calculated as in Eq. 14 or Eq. 16, depending on the experimental geometry. Then, we skip the interpolation and simply do the angle-to-momentum conversion, which is typically done for ARPES experiments. The warp is done forwards and backwards to show that the original k-mesh is reproduced.

To convert from angle to momentum, we simply define a transformation matrix T consisting of all the rotational matrices:

$$\begin{aligned} T &= R_{\text{motors}}(\theta_m, \phi_m) R_{\text{offset}}(\theta_0, \phi_0, \alpha_0) \\ &= R(\theta_m, \hat{y}) R(\phi_m, \hat{x}) R(\theta_0, \hat{y}) R(\phi_0, \hat{x}) R(\alpha_0, \hat{z}) \end{aligned} \quad (17)$$

Then the normalized vector for photoelectron k_{norm} is given by solving $Tk_{\text{norm}} = d$

$$k = |k|T/d, \quad (18)$$

where $d = (0, 0, 1)$, and $|k| = \sqrt{2m_e E_k} \hbar$. This function doesn't generate any outputs, but plots the intensity before warping, after warping to angle, and after warping back to momentum space.

1.3.4 Creating statistics

```
def Make_statistics(self, exp):
```

This function takes the ARPES intensity $I(\theta, \phi, \omega)$ and uses it as a probability distribution function to generate a measured spectrum of N electrons. The inputs are:

- `exp`: An object containing the experimental parameters, such as the background, detector specifications, number of electrons, and more. Details are given in Sec. 2.3.

First, we create the probability distribution function:

$$p = (I_{\text{arpes}} + I_{\text{bgd}}) \times R_{\text{detector}} \quad (19)$$

To generate the random electrons, we reshape p into a one dimensional vector and turning the intensity into bin-edges of a histogram using a cumulative sum. The bin-edges are then normalized so that it is on the interval $[0, 1]$. Following this, we generate N_e random numbers on the interval $[0, 1]$, and count the number of elements in each bin (N_m). As the bin sizes correspond to the probability. If offset domains have been calculated, the same procedure is applied to `arpes.domains` to generate a statistical simulation.

The outputs of the function are:

- `self.stats`: A numpy array containing the spectra generated by N_e random electrons that has the same dimension as `self.intensity`.
- `self.dstats`: A numpy array containing the spectra generated by N_e random electrons that has the same dimension as `self.domains`.

1.3.5 Quality scoring

```
def Make_quality_score(self, param = None):
```

One of the goals of the deep-learning algorithm is to be able to assess the quality of the ARPES data in real time, to determine the best alignment and best spot to measure the sample. Therefore, it is important to create a scoring index for the quality of the data and the simulation. This function provides a quality score for the spectra based on the metrics, feature sharpness, the number of electrons, the signal to background, and a total score.

The quality score starts at a maximum of 10 points. First, each of the metrics (sharpness, signal to background, and number of electrons) are scored on an interval of 0 to 1, with 1 being highest and 0 being lowest. Then, depending on the score, penalties and bonuses are added to the quality score.

The input `param` is a dictionary of the form

`param = {"threshold": [], "penalty": [], "weight": []}`. The fields are:

- "threshold": A numpy array of dimension $m \times 1$ defining the score brackets where each penalty is incurred. The default is [0.9, 0.7, 0.5, 0.3].
- "penalty": A numpy array of dimension $m \times 1$ defining the penalty amount incurred in each score bracket. The default is [1, 2, 3, 4].
- "weight": A numpy array of dimension 3×1 defining the relative weights of penalties for each metric. The default is [1, 1, 1].

Additionally, if the background score (S_{bkgd}) and the electron number score (S_{Ne}) are both very low ($S_{\text{bkgd}} + S_{\text{Ne}} < 0.5$), the score incurs an additional penalty of -2. If S_{bkgd} or S_{Ne} is greater than 0.9, the score gains a bonus of 1. The outputs of the function are:

- `self.score`: A dictionary with the fields:
 - "width": The feature sharpness score, out of 1
 - "bkgd": The signal to background score, out of 1
 - "counts": The number of electrons score, out of 1
 - "final": The total score, out of 10.

1.3.6 Print variables

```
def print_variables(self):
```

Provides a summary of variables used to calculate the measured arpes spectra.

2 Module 2: aurelia_static_vars

While the first module contains the objects of the simulation, the second module contains the parameters informing the simulation. Three objects are defined here:

- The modifier `mod` contains parameters such as resolution broadening for modifying the observed spectral intensity.
- The `domain` object contains parameters regarding the presence of rotational or flake domains.
- The experiment `exp` object contains the parameters pertaining to experimental conditions that may affect the measured spectra, from fundamental parameters such as the photon energy, to more technical parameters such as the signal to background and detector responsivity.

2.1 Class: Modifier

2.1.1 Initialization

```
def __init__(self, temperature=None, resolution=None):
```

The inputs of the function are as follows:

- `temperature`: A float defining the electronic temperature. If not given, it is a random value between 10 and 350. (Lower values can be given but the number of points in the energy dimension (ω) the calculation must increase for it to be accurate, increasing computational time).
- `resolution`: A python dictionary of the form `resolution = {"ER": [], "kR": []}`, where ER (kR) are floats defining the energy (momentum/angular) resolution of the setup, respectively.

The outputs of the function are:

- `self.Temp`: A float ,the electronic temperature.
- `self.ER`: A float ,the energy resolution.
- `self.kR`: A float ,the momentum/angular resolution.

2.2 Class: Domain

2.2.1 Initialization

```
def __init__(self, az=None, flake=None):
```

The inputs of the function are as follows:

- `az` is a parameter specifying the the rotational domains. It has the following fields:
 - `az["Num"]`: An integer r determining the number of rotational domains. If it is not specified, this number is chosen randomly on the interval $[1, 3]$.
 - `az["az"]`: A numpy array of dimension $r \times 1$. Contains the azimuth angle of each rotational domain. If it is not specified, the elements are chosen randomly on the interval $[-30, 30]$.
 - `az["amp"]`: A numpy array of dimension $r \times 1$. Contains the relative intensity amplitude of each rotational domain. If not specified, the elements are chosen randomly on the interval $[0, 1]$.
- `flake` is a parameter specifying the the flake domains. It has the following fields:
 - `flake["Num"]`: An integer f determining the number of flake domains. If it is not specified, this number is chosen randomly on the interval $[0, 5]$.
 - `flake["th"]`: A numpy array of dimension $f \times 1$. Contains the θ offset angle of each flake domain. If it is not specified, the elements are chosen randomly on the interval $[-5, 5]$.
 - `flake["ph"]`: A numpy array of dimension $f \times 1$. Contains the ϕ offset angle of each flake domain. If it is not specified, the elements are chosen randomly on the interval $[-5, 5]$.
 - `flake["amp"]`: A numpy array of dimension $f \times 1$. Contains the relative intensity amplitude of each flake domain. If it is not specified, the elements are chosen randomly on the interval $[0, 1]$.

The dictionaries `az` and `flake` are written into the object to carry over into further calculations.

2.2.2 Make rotational domain

`def Make_domain_rot(self, spec):`

This function creates rotational domains in the spectra, which – in real experiments – can result from the growth of the sample, and are often at very precise angles α_0 . Often times, the rotational domains are decoupled from the flake domains (with angles θ_0 and ϕ_0 , which naturally arise during the sample cleaving process).

Computationally, the fastest way of creating rotational domains is to rotate the k-path by the azimuth angle α_0 :

$$\begin{aligned} k_x^{\text{rot}} &= k_x \cos \alpha_0 - k_y \sin \alpha_0 \\ k_y^{\text{rot}} &= k_x \sin \alpha_0 + k_y \cos \alpha_0 \end{aligned} \tag{20}$$

Then, we define an interpolant F using the points $(k_x^{\text{rot}}, k_y^{\text{rot}})$ and the values E_n . To generate the rotated bands E_n^{rot} , we simply interpolate F onto the square grid (k_x, k_y) . The rotated

bands E_n^{rot} are appended to the list of bands E_n . In the same way, we generate matrix elements M_n^{rot} and the self-energy Σ_n for these new rotated bands, which are additionally appended onto `spec.matrix_elements`, `spec.ImS` and `spec.ReS`. The relative intensity of these domains are added during the calculation of `spec.specfun`. The inputs of the function is simply the object `spec`, which contains all the information about the dispersion.

While most of the outputs of the function are appended onto existing fields, there are a few new additional fields, namely:

- `self.type`: A string "rot" indicating a rotational domain is added.
- `spec.domain_info`: The object domain, which contains the parameters specifying the rotational domain.

2.2.3 Remove rotational domains

```
def Rmv_domain_rot(self, spec):
```

This function removes the rotational domains by removing the appended values from the following fields of the object `spec`:

- `Bands.bands`
- `spec.matrix_elements`
- `spec.ReS`
- `spec.ImS`
- `spec.domain`
- `self.type`: The string "rot" is removed

2.2.4 Make flake domain

```
def Make_domain_offset(self, arpes, speed='fast'):
```

This function creates domains originating from sample flakes at different offset angles than the dominant domain. The inputs of the function are:

- `arpes`: An object containing the simulated ARPES intensity and associated parameters.
- `speed`: A string that takes two inputs, "fast" or "slow". The most accurate way of creating offset flakes is changing θ_0 , ϕ_0 , and using `arpes.Make_angle_conv(domain)` to calculate the spectra for each domain. However, the interpolation during momentum-to-angle conversion is slow. Less accurately, we can simply take the spectrum and shift it in θ_m and ϕ_m by θ_0 and ϕ_0 . This will introduce errors on the edges or at large angles. This string allows the user to choose between the two modes. The default mode is "fast".

- `self.flake`: A python dictionary containing the parameters for the flake domain.

The outputs of the function are:

- `self.type`: A string "offset" indicating a flake domain is added.
- `spec.domain_info`: The object domain, which contains the parameters specifying the rotational domain.
- `arpes.domain`: Contains the intensities from the domains that are calculated.

2.2.5 Remove flake domains

`Rmv_domain_offset(self, arpes):`

This function removes the rotational domains by removing the fields of the object `arpes`:

- `arpes.domain`
- `arpes.domain_info`
- `self.type`: The string "offset" is removed.

2.3 Class: Experiment

2.3.1 Initialization

`def __init__(self, spec, angles=None, hv=None, bkgd=None, Ne=None, detector=None):`

The exposed inputs of the function are as follows:

- `spec`: An object containing the spectral intensity calculated from the dispersion.
- `angles`: The offset angles of the primary flake. To specify the angles, input a dictionary of the form `angles={"az0": [], "th0": [], "ph0": []}`. To set all offset angles to zero, input `angles = "zero"`. If no input is given, the offset angles `th0` and `ph0` are randomly determined on the interval $[-10, 10]$. The angle `az0` is randomly selected on the interval $[0, 360]$.
- `hv`: A float specifying the photon energy of the light that induces the photoemission process (in eV). In order for θ and α to be real values (i.e. for the photoelectron to be photoemitted), we need to satisfy the conditions $\hbar|k| \leq \sqrt{2m_e E_k}$, which gives:

$$E_k = h\nu - \Phi - E_b \geq \frac{\hbar^2(k_x^2 + k_y^2)}{2m_e} \quad (21)$$

Here Φ is the work function of the material. Of course this condition must be satisfied for the deepest binding energies ($\max(E_b) = \min(\omega)$) and largest momenta. So this condition becomes:

$$h\nu \geq \frac{\hbar^2 k_{\text{lim}}^2}{m_e} + \min(\omega) + \Phi \quad (22)$$

For these $h\nu$ values, $0^\circ \leq \theta \leq 90^\circ$. However, we ideally would like θ to be collected within 45° or so, so we add an additional factor of 2 to the RHS of Eq. 22. Lastly, we add a random number r in the interval $[0, 5]$ eV to $h\nu$, giving

$$h\nu = \frac{2\hbar^2 k_{\text{lim}}^2}{m_e} + \min(\omega) + \Phi + r \quad (23)$$

- `Ne`: An integer specifying the number of electrons collected. If not given, `Ne` is randomly determined using:

$$\begin{aligned} N_e \in [10^4, 10^6] & \quad \text{if} \quad \text{spec.dimension} = \text{"slicekk"} \\ N_e \in [10^4, 10^7] & \quad \text{if} \quad \text{spec.dimension} = \text{"sliceEk"} \\ N_e \in [10^5, 10^8] & \quad \text{if} \quad \text{spec.dimension} = \text{"cube"} \end{aligned} \quad (24)$$

- `bkgd`: A dictionary specifying the type and strength of the background intensity in the measured spectra. The fields are as follows:
 - `"type"`: A string determining the types of backgrounds. Three inputs are available, and can be used alone or together. If used the combination, enclose the strings in square brackets (i.e. a Python list). The type `"flat"` gives a flat background. The type `"Shirley"` gives a Shirley background. The type `"poly"` gives just a polynomial background.
 - `"flatA"`: A float specifying the amplitude of the flat background relative to the maximum intensity of the spectra. If not given, the value is randomly determined on the interval $[0.01, 0.21]$.
 - `"shirA"`: A float specifying the amplitude of the Shirley background relative to the maximum intensity of the spectra. If not given, the value is randomly determined on the interval $[0.01, 0.11]$.
 - `"polyA"`: A float specifying the amplitude of the polynomial background relative to the maximum intensity of the spectra. If not given, the value is randomly determined on the interval $[0.01, 0.11]$.
 - `"polyOrder"`: An integer specifying the order of the polynomial background. If not given, the value is randomly determined on the interval $[0, 5]$.
- `detector`: A dictionary specifying the parameters of the detector. The fields are as follows

- "response": A string determining the response of the detector. Accepts two inputs, "flat" or "center". The flat response is self explanatory. The center response gives higher responsivity to the center of the detector. "sensitivity": A float determining the difference in sensitivity between the center of the detector (higher) and the edges (lower). Defaults to zero. Only used if `detector["type"] = "center"`.
- "slit": A string determining the orientation of the slit. Accepts two inputs, "horizontal" or "vertical".
- "counting mode": A string determining the counting mode of the detector. Accepts two inputs, "ADC" or "PC". The pulse counting (PC) mode lights up only one electron per pixel, and works better in low electron count modes, while the ADC mode lights up a blob of pixels per electron event. We simulate this by convoluting with a small Gaussian of a few pixels in two dimensions.

The outputs of the function are as follows:

- `self.az0`: A float defining the azimuth offset angle.
- `self.th0`: A float defining the θ offset angle.
- `self.ph0`: A float defining the ϕ offset angle.
- `self.hv`: A float defining the photon energy.
- `self.Ne`: An integer defining the number of electrons.
- `self.bkgd`: A dictionary defining the background parameters.
- `self.detector`: A dictionary defining the detector parameters.

2.3.2 Adding background

```
def Make_bkgd(self, arpes):
```

This function adds various types of background intensity to the ARPES spectra simulation (see Sec. 1.3.4, Eq. 19). The input is the object `arpes`, in which the intensity of the spectra has been calculated.

Here we include a combination of three types of ARPES background noise:

- The Shirley background is calculated using

$$\text{BG}_{\text{Shir}}(\mathbf{k}, \omega) = \int_{\min(\omega)}^{\omega} I(\mathbf{k}, \omega') d\omega', \quad (25)$$

which is then normalized and then multiplied by the amplitude `self.bkgd["shirA"]`.

- The flat background is an array of ones multiplied by the amplitude `self.bkgd["flatA"]`.
- The polynomial background is a 2D polynomial of order `self.bkgd["polyOrder"]`, normalized then multiplied by the amplitude `self.bkgd["polyA"]`. The two dimensions are E_k - θ if `arpes.dimension = "sliceEk"`, and θ - ϕ if `arpes.dimension = "slicekk"`. If `arpes.dimension = "cube"`, then the polynomial is calculated in θ - ϕ and stacked so that it is the same for each energy.

The output of the function is:

- `arpes.bkgd`: A numpy array with the same dimensions as `arpes.intensity` containing the background.

2.3.3 Adding responsivity

```
def Make_detector_responsivity(self, arpes):
```

The function creates the detector responsivity for the ARPES spectra simulation (see Sec. 1.3.4, Eq. 19). The input is the object `arpes`, in which the intensity of the spectra has been calculated. Here we include two types of response.

- The flat response gives an ideal even response.
- The center response is a two-dimensional quadratic, with the highest response at the center and lowest at the edges. The response difference between the center and the edge is given by `self.detector["sensitivity"]`.
- The two dimensions are E_k - θ if `detector["type"] = "HA"`, and θ - ϕ if `detector["type"] = "TOF"`.

The output of the function is:

- `arpes.response`: A numpy array with the same dimensions as `arpes.intensity` containing the response.

3 Module 3: aurelia_plots

3.1 Class: show_spectra

This object contains plotting functions that allows one to check the status of the calculation. There are no outputs to these functions, and they do not change the calculation at all.

3.1.1 Plot spectral function and bands

```
def Make_spec_plot(spec, constE=None):
```

This function plots the values in `spec.specfun` as well as the dispersion in `bands.bands`. The key input of this function are the object `spec`, which can have three dimensions options: "cube", "slicekk", "sliceEk". If the dimension is "cube", one has to pick slices to plot. The input `constE` specifies the constant energy cut to show. By default, the Fermi surface is chosen.

3.1.2 Plot ARPES spectra

```
def Make_arpes_plot(arpes, exp):
```

This function plots the values in `arpes.intensity`. The offset angles θ_0 , ϕ_0 , and α_0 defined in the experiment object `exp` are also shown by red lines to check the momentum-to-angle conversion is correct. If the dimension is "cube", one has to pick slices to plot. By default, the Fermi surface is chosen alongside the middle value of `arpes.th` and `arpes.ph`.

3.1.3 Plot domains

```
def Make_flake_plot(arpes, exp, domain):
```

This function plots the values in `arpes.intensity` and `arpes.domains`. The offset angles θ_0 , ϕ_0 , and α_0 defined by both the experiment object `exp` and the domain object `domain` are shown by red lines to check the correctness. If the dimension is "cube", one has to pick slices to plot. By default, the Fermi surface is chosen alongside the middle value of `arpes.th` and `arpes.ph`.

3.1.4 Plot statistical simulation

```
def Make_stats_plot(arpes):
```

This function plots the values in `arpes.stats` and `arpes.dstats`, if available. If the dimension is "cube", one has to pick slices to plot. By default, the Fermi surface is chosen alongside the middle value of `arpes.th` and `arpes.ph`.

4 Module 4: aurelia_hdf5_saving

This script provides functionality to serialize Python dictionaries and class instances into HDF5 files using the `h5py` library. It supports nested structures, compresses NumPy arrays, and stores primitive types as attributes. These functions are used to save the simulations into `.h5` files.

4.1 Class: DictGroup

A helper class that recursively stores dictionary contents into HDF5 groups and datasets.

Usage:

```
DictGroup(parent, name, data)
```

- `parent`: HDF5 file or group
- `name`: Name of the group
- `data`: Dictionary to store

4.1.1 `convert_strings_to_utf8(obj)`

Recursively converts strings in objects to UTF-8 encoded bytes for HDF5 compatibility. Handles strings, lists, tuples, dictionaries, and objects with `__dict__`.

4.1.2 `save_dict_to_hdf5`

Saves a dictionary into an HDF5 file.

Parameters:

- `dict_obj`: Dictionary to save
- `folderpath`: Path to save the file
- `filename`: Name of the HDF5 file
- `groupname`: Name of the group inside the file

Behavior:

- NumPy arrays are saved as datasets with gzip compression
- Primitive types (int, float, str, etc.) are saved as attributes
- Nested dictionaries are saved as subgroups recursively

4.1.3 `save_class_to_hdf5`

Saves a Python class instance to an HDF5 file.

Parameters:

- `obj`: Class instance
- `folderpath`: Path to save the file

- filename: Name of the HDF5 file
- groupname: Optional name of the group

Behavior:

- Non-callable, non-private attributes are saved
- NumPy arrays are saved as datasets with gzip compression
- Primitive types are saved as attributes
- Dictionaries are saved using `save_dict_to_hdf5`
- Nested objects are saved recursively using `save_class_to_hdf5`

4.1.4 Example file structure

Given a class:

```
class Sample:
    def __init__(self):
        self.name = "Test"
        self.data = np.array([1, 2, 3])
        self.meta = {"version": 1.0, "author": "Alice"}
```

Calling:

```
save_class_to_hdf5(Sample(), "./", "output.h5")
```

Produces:

```
output.h5
Sample
  name (attribute: "Test")
  data (dataset: [1, 2, 3])
  meta
    version (attribute: 1.0)
    author (attribute: "Alice")
```

5 Example code

5.1 Simulating spectra

This is an example piece of code that generates random ARPES spectra in the dimension "slicekk", or, constant-energy contours.

Step 1: Import all the relevant libraries and classes.

```
import numpy as np
from aurelia_arpes import Bands, Spec, ARPES
from aurelia_static_vars import mod, experiment
import aurelia_hdf5_saving as sv
from aurelia_plots import show_spectra as sh
import os
```

Step 2: Define where the data are saved and how many spectra to generate.

```
path = os.path.join(".", "data_QS")
os.makedirs("data_QS", exist_ok=True)
N_runs=1
```

Step 3: Define desired exposed inputs that will inform the simulation.

```
#Number of points in for theta, phi
Npts_in=[500, 150]
#Input matrix elements
ME_in = {"type": ['poly', 'symm', 'rot']}
#Experimental offset angles
exp_offsets={"ph0":np.radians(np.random.uniform(-10,10)),\
             "th0":np.radians(np.random.uniform(-10,10)),\
             "az0":np.radians(np.random.uniform(-5,5))}
#Self energy
SE={"type": 'FL'}
#Background
bg={"type": ('flat', 'poly')}
#Detector settings
det={"response": 'center', "sensitivity": np.random.uniform(0.5,1),\
     "type": 'HA', "slit": "horizontal", "counting mode": "ADC"}
```

Step 4: Run the simulation.

```
for i in range(N_runs):
    #Define bands
```

```

B=Bands(Npts = Npts_in, edges=1.25)
B.Make_kpath()
B.Make_bands()
B.print_variables()
#Calculate spectra (in Eb and k)
spec=Spec(B, ME=ME_in, dimension='slicekk')
spec.Make_matrix_elements()
spec.Make_self_energy(SE)
m=mod()
spec.Make_specfun(m)
spec.Make_specmod(m)
spec.print_variables()
sh.Make_spec_plot(spec)

#Calculate experimental arpes spectra
ang_in={"th":np.deg2rad([-15,15]),\
        "ph":np.deg2rad([-10-np.random.randint(10),\
        10+np.random.randint(10)])})
exp=experiment(spec, detector=det, bkgd=bg, Ne=10**6)
arpes=ARPES(spec, exp, ang_lim=ang_in)
arpes.Make_angle_conv()
sh.Make_arpes_plot(arpes,exp)

#Add experimental artifacts
arpes=exp.Make_bkgd(arpes)
arpes=exp.Make_detector_responsivity(arpes)
arpes.Make_statistics(exp)
sh.Make_stats_plot(arpes)
arpes.print_variables()

#Optional saving
print(i)
filename='arpesFS_'+str(i)+'.h5'
sv.save_class_to_hdf5(arpes,path,filename)

```

Within the for loop, first the dispersion (B) is determined, followed by the calculation of the spectral function (spec), conversion to ARPES spectra (arpes), and finally the statistical simulation. At each step, variables informing the simulation are printed and shown, and relevant plots are shown. At the end, the simulation is saved as an .h5 file.

5.2 Adding domains

For adding offset domains to the simulation, we can add the following code to the steps defined in Sec. 5.1.

Add to Step 1: This imports the domain class.

```
from aurelia_static_vars import domain
```

Add to Step 3: This specifies the number of flakes and the theta and phi angles. Note that here we specifically define the number of flakes and the interval over which the θ_0 offset angle is randomly selected, but allow the ϕ_0 offset angle to be chosen randomly per the default interval of $[-5, 5]$ by not giving a value to `flake["ph"]`. The intensity of the flake domains is also not specified via the field `flake["amp"]`, so the intensity of each flake will be randomly chosen on the interval $[0, 1]$.

```
flake={"Num": 2}
flake["th"] = np.deg2rad(np.random.uniform(-2, 5, flake["Num"]))
d=domain(flake=flake)
```

Note also that though the rotational domains are not explicitly defined here, the parameters informing the rotational angle, number of domains, and amplitude are already randomly generated though the domains are not yet created. We could generate them explicitly, similarly to how we defined the flake domains, but it is not necessary.

To create rotational domains:

Add to Step 4: Input after `spec.Make_self_energy(SE)` and before calculating the spectral function.

```
B, spec = d.Make_domain_rot(spec)
```

To create flake domains:

Add to Step 4: Input after `arpes.Make_angle_conv()` and before creating the background intensity.

```
arpes=d.Make_domain_offset(arpes)
```

Both types of domains can be created at the same time.

5.3 Adding quality score

This functionality was written specifically for the purpose of training a machine-learning algorithm to assess ARPES spectra quality.

To define quality score parameters (optional)

Add to Step 3:

```
score_param={"threshold": [0.9, 0.7, 0.5, 0.3, 0.1, 0], \  
             "penalty": [1, 2, 3, 5, 8], \  
             "weight": [0.6, 1, 0.8]}
```

To create quality score

Add to Step 4: Input after `arpes.Make_statistics(exp)`

```
arpes.Make_quality_score(param = score_param)
```

6 Appendix

6.1 Momentum to angle conversion details:

First, we define the sample's out-of-plane direction as $\hat{z}$, then the axis that is parallel (perpendicular) to the slit of the hemispherical analyzer is denoted $\hat{y}$ ($\hat{x}$). We will define the polar angle θ to be a rotation about the $\hat{y}$ axis, the tilt angle ϕ to be a rotation about the $\hat{x}$ axis, and the azimuth angle α to be about the $\hat{z}$ axis. The θ and α can be calculated from the conservation of momentum:

$$\begin{aligned} k_x &= \frac{\sqrt{2m_e E_k}}{\hbar} \sin \theta \cos \alpha \\ k_y &= \frac{\sqrt{2m_e E_k}}{\hbar} \sin \theta \sin \alpha \end{aligned} \quad (26)$$

Rearranging gives:

$$\begin{aligned} \theta &= \text{sign}(k_x) \arcsin \left(\frac{\hbar \sqrt{k_x^2 + k_y^2}}{\sqrt{2m_e E_k}} \right) \\ \alpha &= \arcsin \left(\frac{\hbar k_y}{\sqrt{2m_e E_k} \sin \theta} \right) \end{aligned} \quad (27)$$

For simplicity, we will start by assuming that the offset angles $(\theta_0, \phi_0, \alpha_0)$ are zero. To find the unit vector specifying the direction of the electron propagation, we can use rotation matrices specified by the angle and rotation axis $R(\theta, \hat{y})$ and $R(\alpha, \hat{z})$. Then the vector defining the direction of photoelectron propagation is:

$$\begin{aligned} d' &= R(\alpha, \hat{z}) R(\theta, \hat{y}) d \\ &= \begin{pmatrix} \cos \alpha & -\sin \alpha & 0 \\ \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{pmatrix} \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} \\ &= \begin{pmatrix} \cos \alpha \cos \theta & -\sin \alpha & \cos \alpha \sin \theta \\ \sin \alpha \cos \theta & \cos \alpha & \sin \alpha \sin \theta \\ -\sin \theta & 0 & \cos \theta \end{pmatrix} \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} \cos \alpha \sin \theta \\ \sin \alpha \sin \theta \\ \cos \theta \end{pmatrix} \end{aligned} \quad (28)$$

Then to get the angles of emission w.r.t. the analyzer, we simply need to find the θ_m and ϕ_m angles needed to map d' onto d , arranged such that we rotate along the analyzer slit (θ_m) first,

then perpendicular to it (ϕ_m).

$$\begin{aligned}
d &= R(\phi_m, \hat{x})R(\theta_m, \hat{y})d' \\
\begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} &= \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \phi_m & -\sin \phi_m \\ 0 & \sin \phi_m & \cos \phi_m \end{pmatrix} \begin{pmatrix} \cos \theta_m & 0 & \sin \theta_m \\ 0 & 1 & 0 \\ -\sin \theta_m & 0 & \cos \theta_m \end{pmatrix} \begin{pmatrix} d'_x \\ d'_y \\ d'_z \end{pmatrix} \\
\begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} &= \begin{pmatrix} \cos \theta_m & 0 & \sin \theta_m \\ \sin \phi_m \sin \theta_m & \cos \phi_m & -\sin \phi_m \cos \theta_m \\ -\cos \phi_m \sin \theta_m & \sin \phi_m & \cos \phi_m \cos \theta_m \end{pmatrix} \begin{pmatrix} d'_x \\ d'_y \\ d'_z \end{pmatrix}
\end{aligned} \tag{29}$$

This then gives rise to a system of three equations with which we solve for θ_m and ϕ_m :

$$\begin{aligned}
0 &= \cos \theta_m d'_x + \sin \theta_m d'_z \\
0 &= \sin \phi_m \sin \theta_m d'_x + \cos \phi_m d'_y - \sin \phi_m \cos \theta_m d'_z \\
1 &= -\cos \phi_m \sin \theta_m d'_x + \sin \phi_m d'_y + \cos \phi_m \cos \theta_m d'_z
\end{aligned} \tag{30}$$

Solving this gives:

$$\begin{aligned}
\theta_m &= -\arctan\left(\frac{d'_x}{d'_z}\right) = -\arctan(\cos \alpha \tan \theta) \\
\phi_m &= \arcsin(d'_y) = \arcsin(\sin \alpha \sin \theta)
\end{aligned} \tag{31}$$

From here, adding the offset angles are rather trivial. We simply apply the rotation matrices $R(\theta_0, \hat{y})$ and $R(\phi_0, \hat{x})$ to d before calculating the photoemission angles. Such that

$$d' = R(\alpha, \hat{z})R(\theta, \hat{y})R(\phi_0, \hat{x})R(\theta_0, \hat{y})d \tag{32}$$

From here we can check that the k-to-angle warping is correct by warping back to k and checking that the results are consistent. To do this, we simply take the angles (θ_m, ϕ_m) , and use them to define the rotation matrices $R(\theta_m, \hat{y})$, $R(\phi_m, \hat{x})$, and define the transformation matrix $T = R(\phi_m, \hat{x})R(\theta_m, \hat{y})$, then $k = K(T^{-1}d)$, where $K = \sqrt{2mE_k}/\hbar$. To add the offset angles, simply define $T = R(\phi_m, \hat{x})R(\theta_m, \hat{y})R(\alpha_0, \hat{z})R(\phi_0, \hat{x})R(\theta_0, \hat{y})$.

Above, we have applied the rotational matrices along the slit first (θ) and then perpendicular to it (ϕ). Depending on the experimental geometry, the opposite may be true. If this is the case, we simply reverse the order of the rotational matrices to $R(\theta_m, \hat{y})R(\phi_m, \hat{x})$:

$$\begin{aligned}
d &= R(\theta_m, \hat{y})R(\phi_m, \hat{x})d' \\
\begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} &= \begin{pmatrix} \cos \theta_m & 0 & \sin \theta_m \\ 0 & 1 & 0 \\ -\sin \theta_m & 0 & \cos \theta_m \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \phi_m & -\sin \phi_m \\ 0 & \sin \phi_m & \cos \phi_m \end{pmatrix} \begin{pmatrix} d'_x \\ d'_y \\ d'_z \end{pmatrix} \\
\begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} &= \begin{pmatrix} \cos \theta_m & 0 & \sin \theta_m \\ \sin \phi_m \sin \theta_m & \cos \phi_m & -\sin \phi_m \cos \theta_m \\ -\cos \phi_m \sin \theta_m & \sin \phi_m & \cos \phi_m \cos \theta_m \end{pmatrix} \begin{pmatrix} d'_x \\ d'_y \\ d'_z \end{pmatrix}
\end{aligned} \tag{33}$$

And the motor angles θ_m and ϕ_m can be calculated from the components of d' as in Eq. 31. Having obtained this conversion for all k_x and k_y , we can simply define a square mesh of angles θ_M and ϕ_M . and interpolate onto the mesh using the values of $(\theta_m, \phi_m, I_{\text{res}}(\mathbf{k}, \omega))$. The outputs of the function are collected in the structure *Spec*.

- *Spec.A_ang*: An array of dimensions [Npts+10, Npts, Npts]. Elements are the intensity map as a function of θ_M and ϕ_M .
- *Spec.angles* = An array of dimensions [Npts, 2]. The first column is the axis θ_M , and the second column is the axis ϕ_M .
- *Spec.Ek* = An array of dimensions [Npts+10, 1]. Values are the kinetic energy of the photoelectrons.
- *Spec.bkgd* = An array of the dimensions [Npts+10, Npts, Npts]. Values are zeros, this is in preparation for the next step of the simulation.